

VIKTOR CHALLENGE · TUM.AI · BUILD THE ROUTER

Bayern CodeWerk

**–35% of Viktor's bill on a held-out chunk we
had never seen**

Riya Biju · Harsha Sathish · Pavin Sumathi Palanichamy · Prethebha Muthukumaran ·
Rohan Sanjay Patil

744 of 1,000 held-out tasks rerouted · all nine models still in play · zero multi-call chains, verified twice

What the router actually looks at

A price ladder, and named moves

Nine models, \$0.20 to \$10.00 per million input tokens. Low-stakes wording starts at the bottom rung; high-stakes wording starts three rungs up. Then six rules move it on what *actually happened*: a stakes tool really fired **+3** · stuck retrying **+4** · resumed from a background wait **−1**. **Every fired rule is logged**, so any decision reduces to one line.

Checked against data, not intuition

Our first stakes list included *send* and *message*. It fired on **411 of 1,000 tasks** — almost all routine Slack posting, not risk. Narrowing it to genuinely irreversible actions dropped that to **56**, and that fix is what took the router from *worse than baseline* to better than it.

BEFORE WE CLAIM ANYTHING

The bill is \$155, not \$87

The starter kit prices **input text only**. It misses ~4,200 tokens of tool definitions shipped on every call, and it misses output, billed at up to **5× the input rate**. Counted properly, the logged corpus is **\$155.23**, not \$87.42 — **the kit understates its own baseline by 44%**.

We also found a caching bug in it: its grouping hash credits a discount on **35,344 tokens that were never shared**. Leaving that discount in reproduces their printed \$87.42 exactly; removing it gives \$87.52. That is how we know where the error is.

Every percentage on the next slides is a percentage of \$155.23.

THE TEST

Held out, cold

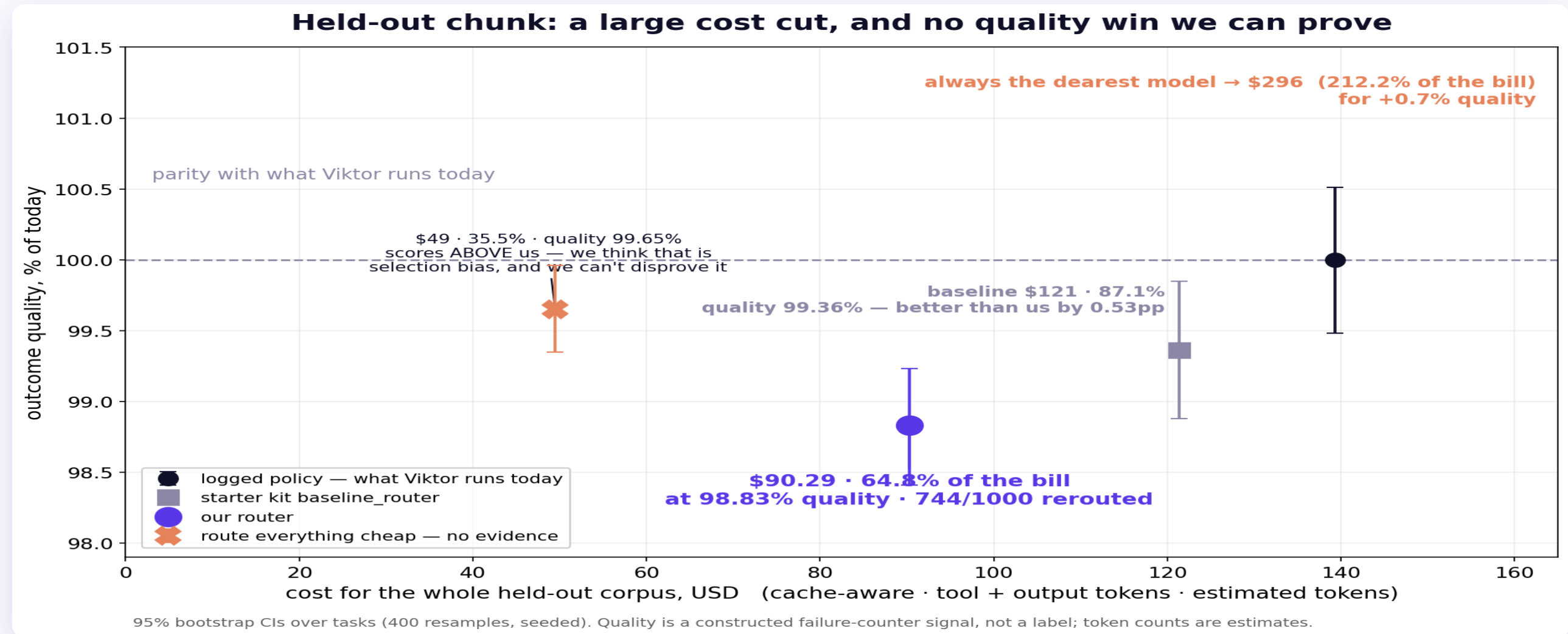
Chunk 02 was released on the final morning. The policy was **fitted on chunk 01 and applied cold** — nothing re-tuned.

	build	held-out
logged	\$155.23	\$139.32
our router	\$97.16 (62.6%)	\$90.29 (64.8%)
rerouted	775 / 1000	744 / 1000
models used	9	9

–37% on the build split, –35% held out. Within two points. It generalises, it doesn't collapse onto the cheap models, and both chunks together contain **zero multi-call chains** across 2,000 rows — independently confirming the organisers' ruling.

THE ONE CHART

Cost–quality frontier (cache-aware · 95% bootstrap CIs)



Quality is plotted as % of what Viktor gets today. Held-out: **\$90.29 at 98.83%**, CI [98.41, 99.23]. Starter baseline \$121.35 at 99.36%.

Always-dearest \$295.57 — **212% of the bill for +0.7% quality**.

What generalised, and what didn't

Held — the cost cut. 62.6% → 64.8% of the logged bill, on data the router had never seen. That claim we stand behind.

Didn't — the quality win. On the build split we were **+0.36pp** over the baseline, but the interval $[-0.20, +0.93]$ **spans zero**, so it was never real. Held out it is **−0.53pp**, CI $[-0.90, -0.13]$, which **excludes** zero. The baseline is genuinely a little better on quality, for 22 more points of the bill.

And the one we won't bury — on our own estimator, **routing everything to the cheapest model scores higher than we do** (99.65% vs 98.83%). We think that is selection bias — cheap models look good because the operator only trusted them with easy work — but **we cannot prove it with this data. So we are not claiming a quality win.**

CLOSE

A cost win we can reproduce. Not a quality win we can't.

**\$139.32 → \$90.29 held out · 64.8% of the bill at 98.8%
of current quality**

Four failure modes, named: a fluent, confident *wrong* answer with no failed tool call is invisible to us · our CI overlaps the cheap-everything loophole · counterfactual quality is inherited from similar tasks, not observed · token counts are estimates with a vendor-correlated bias.

Reproducible offline in ~4 minutes. No GPU, no API key. `./evaluation/run.sh export/`